บทที่ 12

ระบบยืนยันตัวตนและการเข้าถึงข้อมูลด้วย Supabase Authentication

การสร้างแอปพลิเคชันที่มีการระบุตัวตน เช่น การรักษาความปลอดภัยสมุดจดบันทึก หรือการจำกัดสิทธิ์ให้เฉพาะเจ้าของบัญชีเข้าถึงข้อมูล จำเป็นต้องใช้ระบบยืนยันตัวตน (Authentication) ในบทเรียนนี้เราจะศึกษาการเชื่อมโยงระบบหลังบ้านของ **Supabase Auth** เข้ากับหน้าจออินเทอร์เฟซในแอปพลิเคชัน React Native Expo SDK 54 ตั้งแต่การจัดการเซสชันผู้ใช้แบบตรงไปตรงมาในจุดเดียว ไปจนถึงการเก็บรักษาความปลอดภัยของโทเค็นด้วยฮาร์ดแวร์เก็บข้อมูลลับ (`expo-secure-store`) และการต่อยอดไปสู่ระบบแชร์สเตตัสระดับสากลด้วย `useContext`

12.1 แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์ (Authentication vs Authorization)

ในการเขียนโปรแกรมระบบความปลอดภัย มีสองแนวคิดที่เราต้องทำความเข้าใจแยกจากกัน:

1. **การยืนยันตัวตน (Authentication - AuthN):** คือกระบวนการพิสูจน์ทราบว่า *"ผู้ใช้งานคือใครจริงหรือไม่"* เช่น การกรอกอีเมลและรหัสผ่าน จากนั้นหลังบ้านจะส่งโทเค็นระบุตัวตนยืนยันกลับมา (เช่น JWT - JSON Web Tokens)
2. **การอนุญาตสิทธิ์ (Authorization - AuthZ):** คือกระบวนการดักกรองว่า *"ผู้ใช้งานที่ยืนยันตนเข้ามาแล้วนั้น มีสิทธิ์ทำอะไรได้บ้าง"* เช่น ผู้ใช้งานทั่วไปเข้าถึงได้เฉพาะตารางของตัวเอง ส่วนผู้ใช้ที่เป็นแอดมินสามารถเข้าดูและลบแถวข้อมูลของทุกคนได้ (ทำงานร่วมกับระบบ RLS ของ PostgreSQL ใน Supabase)

ในการทำระบบล็อกอินบนโทรศัพท์มือถือ อุปกรณ์ไคลเอนต์จะได้รับ Access Token และ Refresh Token จากระบบคลาวด์หลังบ้านมาเก็บรักษาไว้ และต้องส่งแนบโทเค็นนี้ไปกับ API Request ทุกครั้งเพื่อยืนยันตัวตน

12.2 สถาปัตยกรรมระบบของ Supabase Auth (GoTrue Service)

Supabase มีระบบที่พัฒนาขึ้นจากบริการหลังบ้านแบบ Open Source ชื่อ **GoTrue API** (Supabase, 2024) ซึ่งคอยจัดสรรและประมวลผลข้อมูลผู้ใช้อยู่เบื้องหลัง โดยมีฟังก์ชันการยืนยันตัวตนที่หลากหลาย เช่น Email/Password, Magic Link, และ OAuth (เช่น Sign in with Google หรือ Apple)

ในบทเรียนนี้เราจะมุ่งเน้นไปที่การระบุตัวตนที่คลาสสิกและสำคัญที่สุดคือ **Email and Password Authentication** ซึ่งแอปจะคุยตรงกับเซิร์ฟเวอร์ GoTrue ผ่าน Supabase Client SDK

12.3 การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ (Sign Up & Sign In)

ฟังก์ชันการทำงานพื้นฐานประกอบด้วยการลงทะเบียนบัญชีใหม่ และการล็อกอินเข้าใช้งานผ่านชุดคำสั่ง Asynchronous ต่อไปนี้:

1. การสมัครสมาชิกใหม่ (Sign Up)

เมธอด `signUp` จะรับค่าเป็นออบเจกต์ที่มีคีย์ `email` และ `password` เพื่อนำไปจัดสร้างแถวบัญชีใหม่บน Supabase:

```
const handleSignUp = async () => {
  const { data, error } = await supabase.auth.signUp({
    email: 'user@example.com',
    password: 'securepassword123',
  });

  if (error) {
    console.error('สมัครสมาชิกขัดข้อง:', error.message);
  } else {
    console.log('สร้างบัญชีสำเร็จแล้ว:', data.user);
  }
};
```

2. การลงชื่อเข้าใช้งาน (Sign In)

เมธอด `signInWithPassword` จะระบุค่าอินพุตเพื่อกู้คืนสเตตัสการเชื่อมต่อและดึงสิทธิ์โทเค็นกลับมา:

```
const handleSignIn = async () => {
  const { data, error } = await supabase.auth.signInWithPassword({
    email: 'user@example.com',
    password: 'securepassword123',
  });

  if (error) {
    console.error('ล็อกอินไม่ผ่าน:', error.message);
  } else {
    console.log('ยินดีต้อนรับ เข้าสู่ระบบสำเร็จ:', data.session);
  }
};
```

12.4 การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ (Session State & onAuthStateChange)

เพื่อให้ตัวแอปพลิเคชันทราบความเคลื่อนไหวตลอดเวลาว่าปัจจุบันผู้ใช้กำลังเชื่อมต่ออยู่หรือล็อกเอาต์หลุดระบบไปแล้ว เราจะใช้เครื่องมือดักฟังความเคลื่อนไหวเซสชันของ Supabase ที่ชื่อว่า **`onAuthStateChange()`**

คำสั่งนี้เสมือนเป็นสตรีมข้อมูล (Subscription Stream) ที่จะคอยส่งสัญญาณทริกเกอร์เรียกฟังก์ชัน Callback ทำงานทุกครั้งเมื่อสเตตมีการขยับเปลี่ยน (เช่น `INITIAL_SESSION`, `SIGNED_IN`, `SIGNED_OUT`, `TOKEN_REFRESHED`)

ตัวอย่างการคุมสเตตแบบง่ายในระดับ Component (Direct State Monitoring):

เราสามารถใช้ `useState` บันทึกค่าเซสชันไว้ในหน้าจอหลัก และใช้ `useEffect` ดักฟังเหตุการณ์เพื่อสลับ UI ในหน้าเดียวได้อย่างง่ายดาย:

```
import React, { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator } from 'react-native';
import { supabase } from '../lib/supabase';

export default function RootScreen() {
  const [session, setSession] = useState<any>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
  // 1. ดึงเซสชันเริ่มต้นที่อาจค้างอยู่ออกมาดูก่อน
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setIsLoading(false);
    });

  // 2. สมัครรับสัญญาณคอยฟังเมื่อสเตตล็อกอินขยับเปลี่ยน
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
    });

  // 3. คืนค่าแรมยกเลิกสมัครสตรีมเมื่อคอมโพเนนต์ถูกถอดถอน
    return () => {
      subscription.unsubscribe();
    };
  }, []);
```

```
  if (isLoading) {
    return <ActivityIndicator size="large" />;
  }

 // ทำ Conditional Rendering สลับหน้าแบบตรงไปตรงมา
  return session ? <Dashboard session={session} /> : <LoginForm />;
}
```

12.5 การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store

ตามค่าเริ่มต้นของ Supabase JS Client ในโปรเจกต์ทั่วไปมักจะเก็บโทเค็นของระบบลงใน **AsyncStorage** ของเครื่องมือถือ แต่มีข้อเสียร้ายแรงด้านความปลอดภัยคือ **AsyncStorage เก็บข้อมูลไว้เป็นเพียงไฟล์ข้อความ Plaintext JSON ธรรมดาบนเครื่อง** ซึ่งไม่มีการเข้ารหัสข้อมูล (Encryption) ทำให้โทรศัพท์ที่ผ่านการ Root หรือ Jailbreak สามารถแอบแกะโทเค็นไปสวมสิทธิ์แทนเจ้าของได้

สำหรับโมบายแอปพลิเคชันยุคใหม่ จึงบังคับให้นำโทเค็นข้อมูลลับเหล่านี้ไปเก็บไว้ในช่องเก็บข้อมูลลับที่มีระบบเข้ารหัสความปลอดภัยระดับฮาร์ดแวร์ (Keychain สำหรับ iOS และ Keystore สำหรับ Android) โดยเราจะใช้ไลบรารี `**expo-secure-store**` (Expo, 2024)

1. การติดตั้ง Secure Store:

```
npx expo install expo-secure-store
```

2. การสร้างคลาสเก็บข้อมูลความปลอดภัยเชื่อมต่อ Supabase Client:

เราสามารถดีไซน์อ็อบเจกต์ทำหน้าที่เป็น Storage Adapter เพื่อบังคับให้โมดูล Auth ของ Supabase เก็บโทเค็นลงใน Secure Store แทน AsyncStorage:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';
import * as SecureStore from 'expo-secure-store';

// 1. สร้างคลาสจัดการ Secure Store Adapter
const ExpoSecureStoreAdapter = {
  getItem: (key: string) => {
   return SecureStore.getItemAsync(key);
  },
  setItem: (key: string, value: string) => {
    return SecureStore.setItemAsync(key, value);
  },
  removeItem: (key: string) => {
    return SecureStore.deleteItemAsync(key);
```

```
  },
};

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

// 2. ประกาศเชื่อมโดยส่ง adapter เข้าไปใน config
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: ExpoSecureStoreAdapter,
    autoRefreshToken: true,    // เปิดระบายโทเค็นใหม่ให้อัตโนมัติเบื้องหลัง
    persistSession: true,        // บังคับจำค่าเซสชันข้ามการเปิดปิดแอป
    detectSessionInUrl: false,
  },
});
```

12.6 การรีแฟกเตอร์ไปใช้ useContext และ AuthContext

แม้ว่าการจัดเก็บสเตตัสล็อกอินใน Component เดียวจะเข้าใจง่าย แต่หากเราขยายแอปพลิเคชันจนมีหลายหน้าจอ (เช่น หน้าโปรไฟล์, หน้าตั้งค่า, หน้าแก้ไขข้อมูล) ทุกหน้าจอจำเป็นต้องทราบค่า Session ล็อกอินปัจจุบัน ซึ่งการส่ง Prop ต่อกันไปเป็นทอด ๆ (Prop Drilling) จะสร้างความยุ่งยากอย่างมหาศาล

แนวทางปฏิบัติที่ดีที่สุดคือการยกระดับตรรกะมาเก็บไว้ใน `AuthContext` เพื่อแชร์ข้อมูล Session ให้กับทุกหน้าจอทั่วแอปพลิเคชัน

ตัวอย่างการสร้าง context/AuthContext.tsx

```
import React, { createContext, useState, useEffect, useContext } from
'react';
import { supabase } from '../lib/supabase';
import { Session, User } from '@supabase/supabase-js';

interface AuthContextType {
  session: Session | null;
  user: User | null;
  isLoading: boolean;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType>({
  session: null,
  user: null,
  isLoading: true,
  signOut: async () => {}
});
```

```
export const AuthProvider = ({ children }: { children: React.ReactNode })
=> {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
  // เช็คเซสชันเริ่มต้น
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

  // ดักฟังการอัปเดตสเตตัสเซสชัน
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    return () => subscription.unsubscribe();
  }, []);

  const signOut = async () => {
    await supabase.auth.signOut();
  };

  return (
    <AuthContext.Provider value={{ session, user, isLoading, signOut }}>
      {children}
    </AuthContext.Provider>
 );
};

// สร้าง Custom Hook เพื่อความสะดวกในการดึงข้อมูล
export const useAuth = () => useContext(AuthContext);
```

เมื่อใช้แนวทางนี้ ในไฟล์หน้าจอ Layout หลัก (`app/_layout.tsx`) หรือคอมโพเนนต์อื่น ๆ เราจะเรียกใช้งานเพื่ออ่านและทริกเกอร์ล้างสเตตัสเพียงบรรทัดเดียวได้ทันที:

```
const { user, signOut } = useAuth();
```

สรุปท้ายบทเรียน

แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์: ระบบความปลอดภัยของแอปพลิเคชันแบ่งเป็น 2 ส่วนหลักคือ การยืนยันตัวตน (Authentication) เพื่อระบุตัวตนและตรวจสอบว่าผู้ใช้คือใครจริงหรือไม่ และการอนุญาตสิทธิ์ (Authorization) เพื่อตรวจสอบสิทธิ์การเข้าถึงข้อมูลตามขอบเขตที่รับรองไว้ ซึ่งในระบบสมาร์ทโฟนไคลเอนต์จะได้รับ Access Token และ Refresh Token จากระบบคลาวด์เพื่อใช้แนบไปกับ API Request ในการยืนยันตัวตนเสมอ

สถาปัตยกรรมระบบของ Supabase Auth: Supabase Auth ทำงานโดยยึดรากฐานหลักผ่านระบบหลังบ้าน GoTrue API ซึ่งเป็นซอฟต์แวร์บริการที่เปิดโอกาสให้ผู้พัฒนาจัดการข้อมูลสมาชิกและสิทธิ์การเข้าถึงได้อย่างเป็นระบบ โดยรองรับทั้งรูปแบบ Magic Link, การล็อกอินผ่านโซเชียลมีเดียชื่อดัง (OAuth) หรือระบบดั้งเดิมอย่างการล็อกอินแบบใช้อีเมลและรหัสผ่าน (Email/Password Auth) ที่เป็นมาตรฐานสากล

การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ: การเริ่มเปิดระบบบัญชีสมาชิกใช้เมธอด supabase.auth.signUp() โดยการส่งตัวแปรข้อมูลของอีเมลและรหัสผ่านเพื่อบันทึกข้อมูลสมาชิกใหม่ขึ้นตารางเก็บข้อมูลหลังบ้าน และใช้เมธอด supabase.auth.signInWithPassword() เพื่อสแกนตรวจสอบความถูกต้องและกู้คืนสิทธิ์เซสชันของข้อมูลสมาชิกนั้นกลับมาแสดงผลที่หน้าจอแอปพลิเคชัน

การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ: เพื่อให้ตัวแอปพลิเคชันตื่นตัวและเปลี่ยนหน้าต่างอินเทอร์เฟซได้อย่างไหลลื่นตามสถานะผู้ใช้ปัจจุบัน เราจะใช้ตัวดักฟังความเคลื่อนไหวเซสชันของ Supabase ที่ชื่อว่า onAuthStateChange() ในการคอยฟลักซ์สัญญาณส่งข้อมูลกลับมา (Callback) เมื่อตรวจพบการสลับหน้า ข้ามเซสชัน หรือล็อกเอาต์ระบบ เพื่อควบคุมสถานะตัวแปรและวาดหน้าจอใหม่ได้อย่างถูกต้อง

การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store: การเก็บบันทึกข้อมูลโทเค็นความปลอดภัยลงใน AsyncStorage มีความเสี่ยงเนื่องจากไม่มีการเข้ารหัสข้อมูล (Plaintext) ผู้พัฒนาโปรเจกต์เชิงพาณิชย์จึงบังคับใช้ expo-secure-store ในการเข้ารหัสข้อมูลระดับลึกผ่านระบบฮาร์ดแวร์ Key Store ของระบบปฏิบัติการ โดยเราสามารถจัดทำ ExpoSecureStoreAdapter มาต่อเชื่อมกับตัวแอป Supabase client ได้เพื่อความปลอดภัยขั้นสูงสุด

การรีแฟกเตอร์ไปใช้ useContext และ AuthContext: เมื่อโครงสร้างแอปพลิเคชันใหญ่ขึ้น การส่งผ่านค่าเซสชันข้อมูลลงไปทีละหน้าจอ (Prop Drilling) จะยุ่งยากและไร้ประสิทธิภาพ การรีแฟกเตอร์โครงสร้างสเตตล็อกอินหลักด้วยเครื่องมือ React Context API อย่างเช่น

useContext และ AuthProvider จะช่วยแก้ปัญหาโดยทำหน้าที่กระจายสัญญาณส่งมอบข้อมูลสิทธิ์ไปหาคอมโพเนนต์ย่อยทั่วทั้งแอปพลิเคชันได้โดยตรง

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับหลักสถาปัตยกรรมความปลอดภัยของแอปพลิเคชันมือถือ, การใช้คำสั่ง API ของ Supabase Auth สำหรับการสมัครสมาชิกและเข้าสู่ระบบ, การควบคุมสถานะการใช้งาน (Session Lifecycle), การรักษาความปลอดภัยระดับอุปกรณ์ด้วย `expo-secure-store` และแนวคิดของการจัดสร้างโครงสร้างตัวเก็บสเตตัสส่วนกลางด้วย React Context API

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดนิยามความแตกต่างระหว่างแนวคิด "การยืนยันตัวตน (Authentication)" และ "การอนุญาตสิทธิ์ (Authorization)" ได้อย่างถูกต้องที่สุด?

ก. Authentication คือการตั้งข้อจำกัดขนาดกว้างยาวของหน้าจอแอป ส่วน Authorization คือการควบคุมความสว่างของหน้าเลนส์กล้อง

ข. Authentication คือการตรวจสอบพิสูจน์ตนว่าผู้ใช้คือใครจริงหรือไม่ (เช่น ล็อกอินผ่านอีเมล) ส่วน Authorization คือการดักกรองเงื่อนไขว่าผู้ใช้นั้นมีสิทธิ์เข้าถึงหรือแก้ไขข้อมูลชิ้นใดบ้าง (เช่น กฎ RLS)

ค. Authentication คือการเขียนสไตล์จัดกลุ่มกล่อง Flexbox ส่วน Authorization คือการส่งคีย์ผ่านทางสายเครือข่ายอินเทอร์เน็ต

ง. ทั้งสองแนวคิดคือเรื่องเดียวกันและไม่มีส่วนใดแตกต่างกันในทางสถาปัตยกรรมระบบคอมพิวเตอร์

ข้อ 2: บริการหลังบ้านตัวใดที่รันทำหน้าที่ควบคุมกลไกความปลอดภัยและระบบสมาชิก (Authentication Engine) อยู่เบื้องหลังของ Supabase?

ก. Firebase Auth

ข. PostgreSQL Engine

ค. GoTrue API

ง. Auth0 Core

ข้อ 3: หากต้องการเขียนโค้ดบันทึกสร้างบัญชีผู้ใช้งานคนใหม่ด้วยอีเมลและรหัสผ่านเข้าสู่ดาต้าเบส จะต้องเรียกใช้เมธอด Asynchronous ใดของ Supabase JS Client?

ก. `supabase.auth.createUser()`

ข. `supabase.auth.signUp()`

ค. `supabase.auth.register()`

ง. `supabase.auth.insertUser()`

ข้อ 4: เมธอดล็อกอินใดของ Supabase Auth ที่มีบทบาทใช้ส่งอีเมลและรหัสผ่านเพื่อเรียกคืนข้อมูลสิทธิ์และรับเซสชันโทเค็นปัจจุบัน?

ก. `supabase.auth.signIn()`

ข. `supabase.auth.loginWithPassword()`

ค. `supabase.auth.signInWithPassword()`

ง. `supabase.auth.authenticate()`

ข้อ 5: เมื่อต้องการเฝ้าติดตามรับฟังการเปลี่ยนผ่านระดับความเคลื่อนไหวเซสชัน (เช่น เมื่อล็อกอินสำเร็จ หรือล็อกเอาต์หลุดระบบ) ควรเลือกรับสัญญาณจากคำสั่งใดของ Supabase?

ก. `supabase.auth.getSession()`

ข. `supabase.auth.onAuthStateChange()`

ค. `supabase.auth.watchSession()`

ง. `supabase.auth.getUser()`

ข้อ 6: เพราะเหตุใด AsyncStorage จึงถูกมองว่ามีความปลอดภัยต่ำและไม่ควรนำมาใช้เป็นที่เก็บโทเค็นยืนยันตัวตนหลักในโมบายแอปพลิเคชันระดับโปรดักชัน?

ก. เนื่องจาก AsyncStorage เก็บข้อมูลเป็นไฟล์ข้อความธรรมดา (Plaintext JSON) ที่ไม่มีการเข้ารหัสความปลอดภัยระดับเนทีฟ ทำให้อุปกรณ์ที่ถูก Root/Jailbreak สามารถดึงโทเค็นออกไปสวมสิทธิ์ได้ง่าย

ข. เนื่องจาก AsyncStorage ถูกจำกัดความจุเพียง 10 ตัวอักษรเท่านั้น

ค. เพราะคำสั่ง AsyncStorage จะลบข้อมูลทิ้งโดยอัตโนมัติทันทีที่ปิดหน้าจอแอปพลิเคชัน

ง. เนื่องจาก AsyncStorage รองรับเฉพาะการทำงานในระบบ iOS เท่านั้น

ข้อ 7: ไลบรารีใดของ Expo ที่มีหน้าที่ช่วยเข้ารหัสและจัดเก็บรหัสกุญแจสำคัญลงสู่ Keychain/Keystore ของอุปกรณ์มือถือในระดับฮาร์ดแวร์ได้อย่างเสถียรปลอดภัย?

ก. `expo-secure-storage`

ข. `expo-safe-store`

ค. `expo-secure-store`

ง. `expo-keychain`

ข้อ 8: การต่อเชื่อมให้ Supabase Client บังคับหันมาบันทึกและจัดการโทเค็นล็อกอินผ่าน Secure Store แทน AsyncStorage จะต้องส่งอ็อบเจกต์ Adapter เข้าผ่านพร็อพเพอร์ตี้ใดในตอนประกาศตัวสร้าง `createClient`?

ก. `auth.storage`

ข. `auth.secureStore`

ค. `auth.adapter`

ง. `auth.persistSession`

ข้อ 9: ในกรณีที่เราสร้างแอปพลิเคชันขนาดเล็กแบบ Single-page โดยใช้สเตตล็อกอินในหน้านั้นหน้าเดียว วิธีการสลับหน้าจอระหว่าง "หน้าฟอร์มเข้าสู่ระบบ" และ "หน้าแผงควบคุม Dashboard" ใช้วิธีใดที่ง่ายและกระชับที่สุด?

ก. การเปิดกล่อง Modal ทับไว้ตลอดเวลา

ข. การใช้งาน Conditional Rendering ผ่านการดักตัวแปรเซสชัน เช่น `session ? <Dashboard /> : <LoginForm />`

ค. การบังคับยิงคำสั่งเปลี่ยน Router ดีดข้ามโฟลเดอร์หน้าจอโดยออโต้

ง. การสลับหน้าจอด้วยการเปลี่ยนพิกัดความกว้างในสไตล์ชีท

ข้อ 10: หากต้องการขยายแอปพลิเคชันให้ใหญ่ขึ้น (Multi-page) และประสงค์จะแชร์เซสชันผู้ใช้อวาตาร์รวมถึงสิทธิ์การเข้าถึงให้ทุกหน้าจอรับรู้ค่าตรงกันทันทีโดยไม่ต้องส่งพร็อพส์ต่อกันเป็นทอด ๆ (Prop Drilling) ข้อใดคือสถาปัตยกรรม React ที่เหมาะสมที่สุดที่จะนำมาประยุกต์ใช้เป็นของแถมพิเศษ?

ก. การทำลูปสตรีมดักในปุ่มกดของทุกหน้าจอ

ข. การใช้งาน React Context API ผ่านการทำ `useContext` และ `AuthContext.Provider`

ค. การนำข้อมูลเซสชันไปจัดบันทึกฝากไว้ในตัวแปร Global Windows Node

ง. การอัดบันทึกพิกัดตำแหน่งลงใน API พิกัดตำแหน่ง GPS

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ค. และ ง. ผิด:** ข้อจำกัดขนาดกล่องเป็นเรื่องของ CSS/Flexbox และระบบทั้งสองมีบทบาทและการจัดเก็บคีย์ที่แตกต่างกันอย่างสิ้นเชิง

• **ข. ถูก:** Authentication คือด่านระบุตัวตนบุคคล (เช่น การกรอกฟอร์มเพื่อแสดงตัวว่าเป็นเจ้าของบัญชีจริง) ส่วน Authorization คือการเช็คว่าบัญชีที่ระบุตนเข้ามาแล้วนั้นมีสิทธิ์ระดับไหนในการอ่านเขียนแก้ไขแถวข้อมูล (เช่น กฎ RLS ควบคุมตาราง wishlist หลังบ้าน)

เฉลย ข้อ 2: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** Supabase พัฒนาขึ้นมาโดยใช้ PostgreSQL เป็นเบสฐานข้อมูล แต่ตัวที่ทำหน้าที่รันจัดการ Session/Auth โดยตรงคือ GoTrue Service

• **ค. ถูก:** GoTrue คือระบบ API เบื้องหลังที่ทำหน้าที่ควบคุม จัดสรร และระบายโทเค็นความปลอดภัยของตารางข้อมูลผู้ใช้งานบน Supabase Auth

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** คำสั่งอื่นไม่มีระบุไว้ในไวยากรณ์การประกาศฟังก์ชันของจาวาสคริปต์ Supabase Client

• **ข. ถูก:** ไวยากรณ์สมัครสมาชิกใช้งานของ Supabase Auth คือคำสั่ง Asynchronous ชื่อ **`supabase.auth.signUp()`**

เฉลย ข้อ 4: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ในอดีตเคยมี `signIn()` แต่ปัจจุบันใน Supabase JS client v2 ได้ปรับเปลี่ยนมาใช้คำสั่งที่ระบุวิธีการล็อกอินชัดเจนขึ้นเพื่อแยกแยะการใช้เมลกับรหัสผ่าน

• **ข. และ ง. ผิด:** คำสั่งไม่มีระบุใน API

• **ค. ถูก:** คำสั่งที่ถูกต้องในการลงชื่อเข้าใช้ด้วยชุดเมลรหัสผ่านคือ **`supabase.auth.signInWithPassword()`**

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `getSession()` คืนค่าเซสชันของจังหวะนั้น ๆ ครั้งเดียว ไม่ได้รับฟังสัญญาณเรียลไทม์

• **ข. ถูก:** `supabase.auth.onAuthStateChange()` ทำหน้าที่เฝ้าดักฟังและพ่นสัญญาณ Callback ออกมาทุกครั้งที่มีการเปลี่ยนผ่านสเตตล็อกอิน ล็อกเอาต์ หรือการเปลี่ยนเปลี่ยนโทเค็น

• **ค. และ ง. ผิด:** คำสั่งระบุผิดวัตถุประสงค์ หรือไม่มีในสารระบบ Auth API ของ Supabase

เฉลย ข้อ 6: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** AsyncStorage มีความปลอดภัยต่ำเพราะเก็บข้อมูลเป็นตัวอักษรดิบที่แกะดูได้ง่ายบนอุปกรณ์ที่ผ่านการแฮกหรือเข้าถึง root หากมีข้อมูลสำคัญอย่างโทเค็นล็อกอิน ก็เสี่ยงที่จะถูกขโมยสิทธิ์

• **ข. ค. และ ง. ผิด:** AsyncStorage เก็บข้อมูลได้ไม่จำกัดสิทธิ์ (ขีดจำกัดปกติคือ 6MB) และข้อมูลไม่ถูกลบอัตโนมัติเมื่อปิดแอป รวมถึงสามารถรันได้ทั้งบน Android และ iOS

เฉลย ข้อ 7: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** ชื่อสะกดไม่ถูกต้องตามหลักไลบรารีของระบบ Expo

• **ค. ถูก:** ไลบรารีอย่างเป็นทางการในการเข้ารหัสและจัดเก็บคีย์ลับลง Keychain/Keystore บนระดับเนทีฟของ Expo คือ `expo-secure-store`

เฉลย ข้อ 8: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** ในการประกาศสร้าง client ตัวเลือกสำหรับจัดส่ง Storage ตัวใหม่คือระบุเข้าที่พร็อพเพอร์ตี้ `auth.storage` ภายใต้ออบเจกต์ config ของ `createClient`

• **ข. ค. และ ง. ผิด:** ตัวเลือกชื่ออื่นสะกดผิดไวยากรณ์การรับพารามิเตอร์ของ Supabase

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** วิธีอื่นระบุคำสั่งควบคุมซับซ้อน หรือส่งผลเสียต่อการรันสเตตของ React

• **ข. ถูก:** การทำ **Conditional Rendering** ด้วยการประเมินค่า `session ? <Dashboard /> : <LoginForm />` เป็นแนวทางที่ดีและสั้นที่สุดในการสลับภาพหน้าจอในหน้าเดียว โดยไม่ต้องรันจัดการเรื่อง Navigation หลายระดับ

เฉลย ข้อ 10: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: วิธีการเหล่านั้นไม่ใช่การแชร์สเตตที่เป็นระเบียบตามมาตรฐาน หรือทำให้เกิดผลเสียต่อ Memory Leak

• ข. ถูก: React Context API (`useContext` และ `Provider`) คือสถาปัตยกรรมของ React ที่ดีที่สุดในการจัดแชร์สเตตตรงส่วนกลาง เพื่อแชร์ข้อมูลลงสู่หน้าต่างและคอมโพเนนต์ลูกหลานทั้งหมดโดยตรง

ปฏิบัติการท้ายบทเรียน: การสร้างระบบยืนยันตัวตน Email/Password ด้วย Supabase Auth และ Secure Store

ปฏิบัติการนี้จะฝึกทักษะการสร้างระบบสมัครสมาชิก (Sign Up) และระบบเข้าสู่ระบบ (Sign In) ผ่านคลาวด์ **Supabase Auth** บนโทรศัพท์มือถือ โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"สมุดบันทึกรับล็อกสิทธิ์ความปลอดภัยส่วนตัว (Secure Personal Diary)"** ผู้ใช้จะกรอกอีเมลและพาสเวิร์ดที่ผ่านการ Validation ปลายทางเครื่องเพื่อล็อกอินรับโทเค็นความปลอดภัยที่จะถูกเข้ารหัสปกป้องถาวรในฮาร์ดแวร์เก็บข้อมูลลับด้วย `expo-secure-store` และปิดท้ายปฏิบัติการด้วย **แนวเนื้อหาพิเศษ (Bonus Section)** เพื่อทดลองเรียนรู้วิธีการรีแฟกเตอร์แยกส่วนตรรกะไปเป็นระบบจัดเก็บข้อมูลส่วนกลางแบบแชร์ข้ามจอด้วย `useContext`

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งและตั้งค่าโปรเจกต์ Expo SDK 54 ให้จัดเก็บโทเค็นของ Supabase Auth ลงในระดับฮาร์ดแวร์ด้วย `expo-secure-store` ได้สำเร็จ
2. สามารถเขียนฟอร์มรับอินพุต พร้อมการกรองตรวจสอบอีเมลและรหัสผ่าน (Form Validation) ได้ถูกต้องตามมาตรฐาน
3. สามารถเรียกใช้ API `signUp` และ `signInWithPassword` ในการจัดการสมัครและยืนยันตนผู้ใช้กับระบบคลาวด์ได้สำเร็จ
4. สามารถใช้งาน `onAuthStateChange` ร่วมกับสเตตภายใน Component เพื่อสลับสลับสิทธิ์หน้าจอระหว่าง Login และ Dashboard (Conditional Rendering) ได้สำเร็จ
5. สามารถทำความเข้าใจและอธิบายสถาปัตยกรรมของการยกระดับระบบความปลอดภัยล็อกอินไปใช้โครงสร้างแชร์สเตตกลางแบบ `useContext`

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดวางโครงสร้างระบบไฟล์ดังต่อไปนี้:

```
โครงสร้างไฟล์หลักปฏิบัติการ:
├── .env                                  => คีย์ความปลอดภัย EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── lib/
│   └── supabase.ts                       => กำหนดค่า Supabase Client เชื่อมต่อผ่าน Secure Store Adapter
└── app/
    └── index.tsx                         => หน้าควบคุมฟังก์ชัน สมัคร/ล็อกอิน และ Dashboard จัดการข้อมูลสมุดบันทึกส่วนตัว
```

รายละเอียดเงื่อนไขการทำงาน (Logics):

1. **การยืนยันความถูกต้องฟอร์ม (Form Validation):**
   - ฟิลด์ Password ต้องกำหนดเงื่อนไขบังคับป้อนไม่น้อยกว่า 6 ตัวอักษร
   - ฟิลด์ Email ต้องกรอกอิงตามรูปแบบ Regular Expression ของอีเมลมาตรฐาน หากไม่ผ่านเกณฑ์ ให้ป้ายปุ่มบันทึกเป็นสีเทาและแสดงข้อความแจ้งเตือนสีแดงด้านล่าง
2. **การสมัครสมาชิก (Sign Up Process):**
   - กดทริกเกอร์สร้างผู้ใช้และแสดงผล Alert ยืนยันความสำเร็จ
3. **การเข้าสู่ระบบและคงอยู่ (Sign In & Session Retention):**
   - ทันทีที่ล็อกอินผ่าน ให้นำผู้ใช้สลับหน้าไปที่หน้าจอบันทึกส่วนตัว (Secure Diary Dashboard)
   - เมื่อผู้ใช้ทำการปิดแอปพลิเคชันไป แล้วกดรันเปิดแอปกลับมาใหม่ ระบบต้องทำการดึงสถานะโทเค็นความปลอดภัยที่เก็บสะสมไว้ใน `expo-secure-store` และนำทางผู้ใช้งานตรงเข้ามาที่หน้า Dashboard ทันทีโดยไม่ต้องให้พิมพ์ล็อกอินซ้ำซ้อน
4. **ความประณีตในการรายงาน Error:**
   - ดักจับ Error จาก Supabase API (เช่น กรอกรหัสผิด หรืออีเมลเคยลงทะเบียนไปแล้ว) และแสดงผลเป็นป้ายกล่องแจ้งเตือนความปลอดภัยสีแดงสดที่เข้าใจง่าย

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีที่จำเป็น

เปิดใช้งานเทอร์มินัลพิมพ์รันการติดตั้งคำสั่งต่อไปนี้:

```
npx expo install expo-secure-store
```

ขั้นตอนที่ 2: ติดตั้งและเชื่อม Supabase Client ผ่าน Secure Store

สร้างหรือปรับปรุงไฟล์ `lib/supabase.ts` เพื่อเปลี่ยนให้ Supabase Client บันทึกค่าเซสชันลงในหน่วยความจำเข้ารหัสระดับอุปกรณ์เนทีฟ:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';
import * as SecureStore from 'expo-secure-store';

// คลาสตัวช่วยจัดการ Secure Store
const ExpoSecureStoreAdapter = {
  getItem: (key: string) => {
    return SecureStore.getItemAsync(key);
  },
  setItem: (key: string, value: string) => {
    return SecureStore.setItemAsync(key, value);
  },
  removeItem: (key: string) => {
    return SecureStore.deleteItemAsync(key);
  },
};

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: ExpoSecureStoreAdapter,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

รหัสซอร์สโค้ดฉบับสมบูรณ์นี้ทำงานในไฟล์ **`app/index.tsx`** โดยจะจัดการระบบสมัครสมาชิก เข้าระบบ สลับหน้าจอ และบันทึกโทเค็นความปลอดภัยอย่างครบถ้วนจบในไฟล์เดียว:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
```

```
  TouchableOpacity,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  KeyboardAvoidingView,
  Platform,
  Keyboard,
  TouchableWithoutFeedback
} from 'react-native';
import { supabase } from '../lib/supabase';

export default function SecureDiaryAuthApp() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const [session, setSession] = useState<any>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

 // 1. ตรวจสอบสแกนหาประวัติล็อกอินเดิมที่เก็บไว้ใน Secure Store
  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setIsLoading(false);
    });

  // ดักฟังการอัปเดตสเตตัสล็อกอิน/ล็อกเอาต์
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
      setErrorMessage(null); // เคลียร์ข้อความแจ้งเตือนข้อผิดพลาดเก่า
    });

    return () => subscription.unsubscribe();
  }, []);

 // 2. ตรวจสอบความถูกต้องฟอร์ม (Validation Rules)
  const isEmailValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  const isPasswordValid = password.length >= 6;
  const isFormReady = isEmailValid && isPasswordValid;

 // 3. ฟังก์ชันลงทะเบียนสมัครสมาชิก (Sign Up)
  const handleSignUp = async () => {
    if (!isFormReady) return;
    try {
      setIsSubmitting(true);
      setErrorMessage(null);
      Keyboard.dismiss();
```

```
      const { data, error } = await supabase.auth.signUp({
        email: email.trim(),
        password: password,
      });

      if (error) throw error;

      Alert.alert(
        'สมัครสมาชิกสำเร็จ!',
        'ระบบส่งอีเมลยืนยันตัวตนไปยังที่อยู่ของท่านแล้ว (กรุณาเช็คอินบ็อกซ์หรือโฟลเดอร์สแปม)',
    [{ text: 'ตกลง' }]
   );
   // เคลียร์ฟิลด์รหัสผ่าน
      setPassword('');
    } catch (error: any) {
      setErrorMessage(error.message || 'เกิดข้อผิดพลาดในการลงทะเบียน');
    } finally {
      setIsSubmitting(false);
    }
  };

// 4. ฟังก์ชันเข้าสู่ระบบ (Sign In)
  const handleSignIn = async () => {
    if (!isFormReady) return;
    try {
      setIsSubmitting(true);
      setErrorMessage(null);
      Keyboard.dismiss();

      const { error } = await supabase.auth.signInWithPassword({
        email: email.trim(),
        password: password,
      });

      if (error) throw error;

   // เมื่อล็อกอินผ่าน onAuthStateChange จะดักสลับหน้าจอให้อัตโนมัติ
    } catch (error: any) {
      setErrorMessage(error.message || 'รหัสผ่านหรืออีเมลไม่ถูกต้อง');
    } finally {
      setIsSubmitting(false);
    }
  };

// 5. ออกจากระบบ (Sign Out)
  const handleSignOut = async () => {
    try {
```

```
      setIsLoading(true);
      await supabase.auth.signOut();
    } catch (error) {
      Alert.alert('ข้อผิดพลาด', 'ไม่สามารถลงชื่อออกจากระบบได้');
    } finally {
      setIsLoading(false);
    }
  };

  if (isLoading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังตรวจสอบสิทธิ์เข้าระบบ...</Text>
      </View>
  );
  }

 // --- แผงการเรนเดอร์สลับ UI (Conditional Rendering) ---
  if (session) {
  // 1. หน้าจอหลักเมื่อเข้าสู่ระบบได้สำเร็จ (Secure Diary Dashboard)
    return (
      <SafeAreaView style={styles.safeArea}>
        <View style={styles.dashboardContainer}>
     <View style={styles.headerBlock}>
            <Text style={styles.dashboardTitle}>🔒 Secure Diary</Text>
            <Text style={styles.userEmailLabel}>ผู้ใช้:
{session.user.email}</Text>
          </View>

          {/* กล่องบันทึกข้อมูลส่วนตัวจำลอง */}
          <View style={styles.diaryContentCard}>
            <Text style={styles.diaryCardHeader">📝 บันทึกลับสุดยอดประจำวัน</Text>
            <Text style={styles.diaryCardBody}>
        "วันนี้เราได้ศึกษาความแตกต่างระหว่าง Authentication และ Authorization สำเร็จ รวมถึงการ
เปลี่ยนไปใช้ expo-secure-store แทนคีย์ plain text ข้อมูลทั้งหมดถูกป้องกันอย่างแน่นหนาแล้วบนระบบคลาวด์
..."
            </Text>
            <Text style={styles.securityBadge}>🛡️ เข้ารหัสด้วย
Keystore/Keychain</Text>
          </View>

          <TouchableOpacity style={styles.signOutBtn}
onPress={handleSignOut}>
            <Text style={styles.signOutBtnText">🚪 ลงชื่อออกจากระบบ</Text>
          </TouchableOpacity>
        </View>
      </SafeAreaView>
```

```
  );
  }

 // 2. หน้าจอแบบฟอร์มยืนยันตนเมื่อยังไม่มีเซสชัน (Login & Register Interface)
  return (
    <SafeAreaView style={styles.safeArea}>
      <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
        <KeyboardAvoidingView
          behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
          style={styles.keyboardAvoid}
        >
          <View style={styles.loginContainer}>

            {/* หัวข้อล็อกอิน */}
            <View style={styles.loginHeader}>
              <Text style={styles.loginTitle}>🔑 Private Vault</Text>
              <Text style={styles.loginSubtitle}>กรุณาเข้าสู่ระบบด้วยอีเมลส่วนตัวเพื่อเปิดไดอารี่ลับ</Text>
            </View>

            {/* แผงแสดงข้อผิดพลาด */}
            {errorMessage && (
              <View style={styles.errorAlertBox}>
                <Text style={styles.errorAlertText}>⚠ {errorMessage}</Text>
              </View>
        )}

            {/* ช่องกรอกข้อมูล */}
            <View style={styles.formCard}>
              <Text style={styles.inputTitle}>อีเมล (Email Address)</Text>
              <TextInput
                style={styles.input}
                placeholder="กรอกอีเมล (เช่น user@email.com)..."
                placeholderTextColor="#64748b"
                value={email}
                onChangeText={setEmail}
                keyboardType="email-address"
                autoCapitalize="none"
        />

              <Text style={styles.inputTitle}>รหัสผ่าน (Password - ขั้นต่ำ 6 ตัวอักษร)</Text>
              <TextInput
          style={styles.input}
                placeholder="กรอกรหัสผ่านปลอดภัย..."
                placeholderTextColor="#64748b"
```

```
                value={password}
                onChangeText={setPassword}
                secureTextEntry
                autoCapitalize="none"
            />

            {/* ข้อแนะนำการพิมพ์ฟอร์ม */}
            {!isFormReady && (email.length > 0 || password.length > 0) &&
(
              <Text style={styles.formWarningText}>
                * รูปแบบอีเมลต้องถูกต้อง และรหัสผ่านต้องมีความยาวอย่างน้อย 6 ตัวอักษร
              </Text>
            )}

            {isSubmitting ? (
              <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
            ) : (
              <View style={styles.actionButtonGroup}>
                <TouchableOpacity
                  style={[styles.btn, styles.signInBtn, !isFormReady &&
styles.disabledBtn]}
                  onPress={handleSignIn}
                  disabled={!isFormReady}
                >
                  <Text style={styles.btnText}>🔑 เข้าสู่ระบบ</Text>
                </TouchableOpacity>

                <TouchableOpacity
                  style={[styles.btn, styles.signUpBtn, !isFormReady &&
styles.disabledBtn]}
                  onPress={handleSignUp}
                  disabled={!isFormReady}
                >
                  <Text style={styles.signUpBtnText}>📝 สมัครสมาชิก</Text>
                </TouchableOpacity>
              </View>
            )}
          </View>
        </View>
      </KeyboardAvoidingView>
    </TouchableWithoutFeedback>
  </SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
```

```
    backgroundColor: '#0f172a',
  },
  center: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 12,
    fontSize: 14,
  },
  keyboardAvoid: {
    flex: 1,
  },
  loginContainer: {
    flex: 1,
   padding: 24,
    justifyContent: 'center',
  },
  loginHeader: {
    alignItems: 'center',
    marginBottom: 24,
  },
  loginTitle: {
    fontSize: 26,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  loginSubtitle: {
    fontSize: 13,
    color: '#94a3b8',
    textAlign: 'center',
    marginTop: 8,
    lineHeight: 18,
  },
  errorAlertBox: {
    backgroundColor: '#ef444422',
    borderWidth: 1,
    borderColor: '#ef4444',
    padding: 12,
    borderRadius: 8,
    marginBottom: 16,
  },
  errorAlertText: {
    color: '#f87171',
```

```
    fontSize: 12,
    fontWeight: '600',
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderWidth: 1,
    borderColor: '#334155',
  },
  inputTitle: {
    color: '#cbd5e1',
  fontSize: 12,
    fontWeight: '600',
    marginBottom: 6,
    marginTop: 10,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    color: '#ffffff',
    padding: 12,
    fontSize: 14,
  marginBottom: 10,
  },
  formWarningText: {
    color: '#fbbf24',
    fontSize: 11,
    marginTop: 4,
    marginBottom: 10,
    lineHeight: 16,
  },
  actionButtonGroup: {
    marginTop: 14,
  },
  btn: {
    borderRadius: 8,
    height: 44,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  signInBtn: {
```

```
    backgroundColor: '#38bdf8',
  },
  signUpBtn: {
    backgroundColor: 'transparent',
    borderWidth: 1,
    borderColor: '#38bdf8',
  },
  disabledBtn: {
    backgroundColor: '#475569',
    borderColor: '#475569',
    opacity: 0.4,
  },
  btnText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  signUpBtnText: {
    color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
   paddingVertical: 12,
  },
// ส่วนสไตล์หน้าจอ Dashboard
  dashboardContainer: {
    flex: 1,
    padding: 24,
    justifyContent: 'space-between',
  },
  headerBlock: {
    alignItems: 'center',
    marginTop: 20,
  },
  dashboardTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  userEmailLabel: {
    fontSize: 13,
    color: '#10b981',
    marginTop: 6,
    fontWeight: '600',
    backgroundColor: '#065f4633',
    paddingVertical: 4,
```

```
    paddingHorizontal: 12,
    borderRadius: 12,
  },
  diaryContentCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 40,
  },
  diaryCardHeader: {
    color: '#ffffff',
    fontWeight: 'bold',
   fontSize: 15,
    marginBottom: 10,
  },
  diaryCardBody: {
    color: '#94a3b8',
    fontSize: 13,
    lineHeight: 22,
  },
  securityBadge: {
    color: '#10b981',
    fontSize: 11,
    fontWeight: '600',
    marginTop: 15,
  },
  signOutBtn: {
    backgroundColor: '#ef4444',
    borderRadius: 8,
    height: 46,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 20,
  },
  signOutBtnText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 14,
  },
});
```

5. [เนื้อหาของแถมพิเศษ (Bonus Advanced Section)]

หลังจากผู้เรียนพัฒนาโค้ดแบบ Single-page สำเร็จเรียบร้อยแล้ว ในการสเกลหรือขยายแอปพลิเคชันสู่ระบบที่มีหลายหน้าจอในระบบ Expo Router เราจะประยุกต์ใช้แนวทาง `useContext` ในการแชร์เซสชันล็อกอินข้ามคอมโพเนนต์

ให้นักพัฒนาจัดสรรสร้างโครงสร้างไฟล์เพิ่มเติมดังนี้:

ไฟล์ที่ 1: จัดทำ Provider ที่ `context/AuthContext.tsx`

```
import React, { createContext, useState, useEffect, useContext } from
'react';
import { supabase } from '../lib/supabase';
import { Session, User } from '@supabase/supabase-js';

interface AuthContextType {
  session: Session | null;
  user: User | null;
  isLoading: boolean;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType>({
  session: null,
  user: null,
  isLoading: true,
  signOut: async () => {}
});

export const AuthProvider = ({ children }: { children: React.ReactNode })
=> {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    return () => subscription.unsubscribe();
  }, []);
```

```
  const signOut = async () => {
    await supabase.auth.signOut();
  };

  return (
    <AuthContext.Provider value={{ session, user, isLoading, signOut }}>
      {children}
    </AuthContext.Provider>
 );
};

export const useAuth = () => useContext(AuthContext);
```

ไฟล์ที่ 2: ดักจับความปลอดภัยหน้าจอที่ Root Layout app/_layout.tsx

นำเอา Provider เข้ามาครอบโครงหน้าของแอปพลิเคชัน เพื่อแบ่งกลุ่มเส้นทางและสกรีน ด้วยตัวดัก Route Guard:

```
import React, { useEffect } from 'react';
import { Slot, useRouter, useSegments } from 'expo-router';
import { AuthProvider, useAuth } from '../context/AuthContext';
import { ActivityIndicator, View } from 'react-native';

function InitialLayout() {
  const { session, isLoading } = useAuth();
  const segments = useSegments();
  const router = useRouter();

  useEffect(() => {
    if (isLoading) return;

  // เช็คว่าหน้าจอที่จะเข้าอยู่ในกรอบกลุ่มล็อกอินหรือไม่
    const inAuthGroup = segments[0] === '(auth)';

    if (!session && !inAuthGroup) {
   // หากไม่มีเซสชันและไม่ได้อยู่หน้าล็อกอิน ให้ถีบผู้ใช้ไปหน้าล็อกอิน
      router.replace('/(auth)/login');
    } else if (session && inAuthGroup) {
   // หากมีเซสชันและคาอยู่หน้าล็อกอิน ให้กระโดดข้ามไปหน้าแดชบอร์ดหลัก
      router.replace('/(app)/dashboard');
    }
  }, [session, isLoading, segments]);

  if (isLoading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
```

```
'center' }}>
        <ActivityIndicator size="large" />
      </View>
  );
   }

  return <Slot />;
}

export default function RootLayout() {
  return (
    <AuthProvider>
      <InitialLayout />
    </AuthProvider>
  );
}
```

เอกสารอ้างอิง

Expo. (2024). *SecureStore*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/securestore/

Supabase. (2024). *Supabase Auth with React Native*. จาก https://supabase.com/docs/guides/auth/auth-helpers/react-native

Supabase. (2024). *Supabase JavaScript Auth API Reference*. จาก https://supabase.com/docs/reference/javascript/auth-signup

Supabase. (2024). *User Management with Supabase*. จาก https://supabase.com/docs/guides/auth